

Assignment 2 Report

Datawarehouse 2

Business Intelligence (188.429) – Winter Semester 2025

Group 061

Student A: Tim Greß (12412672) – Student A

Student B: Stefan Merdian (12433732) – Student B

Submitted: November 30, 2025

1 ETL Summary

In the fact table `ft_param_city_month` we aggregate sensor readings to a monthly grain of *month* $\times$ *city* $\times$ *parameter*. The ETL is implemented in several CTE stages.

First, raw reading events are joined with sensor metadata to retrieve the corresponding cities and then mapped to the correct dimension keys. Since the technical keys did not match across tables, we relied on unique city and parameter names and mapped months by checking whether the reading timestamp falls within the boundaries of a row in `dim_timemonth`.

Alert thresholds per parameter are used to assign a numeric alert rank to each reading. For each day we keep only the worst alert level, which allows us to compute monthly exceeded counts and peak alert levels. Missing days are derived by joining to a month-length table and subtracting the number of distinct observation days per (city, parameter).

Finally, alert ranks are mapped back to `dim_alertpeak`, all measures are collected into the fact table, and the fact surrogate key is generated via `ROW_NUMBER()` during insertion.

2 Answers to business questions

Student A

SQL Tasks: 03, 04, 14, 18, 24

MDX Tasks: 03, 04, 05, 07, 18

Hierarchies in MDX were difficult (Q04, Q18).

Using hierarchies in MDX required working with the `DESCENDANTS` operator. This caused hierarchy conflicts and forced us to first learn how MDX hierarchy handling works. In SQL the same logic was much simpler due to having explicit columns.

Filtering with WHERE in MDX (Q03, Q07, Q18).

MDX does not allow filtering using the same dimension tables that appear in the `SELECT` clause, so every filtering step needed a workaround. In SQL this is trivial.

Numeric aggregation is easier in SQL (Q14, Q24).

More complex aggregations such as averages are straightforward in SQL. In MDX (e.g. Q07) these measures needed to be predefined, which is less flexible but results in cleaner expressions.

City-specific multi-quarter breakdowns were hard in MDX (Q18).

In MDX city members had to be manually combined with quarter descendants. In SQL this was just `SUM(...)` `FILTER (WHERE quarter = X)`. In addition, the Atoti Python notebook did not correctly render the quarters as separate columns, whereas the Atoti UI did.

Student B

SQL Tasks: 05, 06, 07, 16, 25

MDX Tasks: 06, 10, 12, 16, 28

Top-N logic in MDX (Q06, Q28).

In MDX we had to combine TopCount and Order to get the Top 10 countries or cities. This is less intuitive than SQL, where the same result is just ORDER BY with LIMIT.

Year filtering via subselects (Q06, Q10, Q16).

Instead of a simple WHERE year = 2024 as in SQL, MDX required a subselect on the Time dimension. Understanding that this restricts the whole cube was a bit confusing at first.

Geo hierarchy and regional filters (Q12).

We could not use the Geo hierarchy both on rows and in the WHERE clause. We had to switch to DESCENDANTS from the region “Eastern Europe” down to the country or city level, which would have been just a simple regional filter in SQL.

Quarter breakdown by param category (Q16).

In SQL the grouping by quarter and param category was straightforward. In MDX we had to find the correct Levels() / DESCENDANTS so that only the four quarters of 2024 appeared as columns.

3 Reflection and Lessons Learned

Student A

Working with MDX was completely new for me and therefore this is where I learned the most. Several queries behaved differently in the Atoti Python notebook compared to the Atoti UI, especially in terms of how results were displayed. The UI’s drag-and-drop builder turned out to be the easiest way to assemble MDX and then copy the generated code.

On the SQL and ETL side, I became more comfortable with DBeaver and with tracing issues through the fact-table pipeline. One clear lesson was that collaboration on a shared ETL is difficult because small naming differences can lead to unnecessary debugging. Overall, the assignment showed me the main differences between MDX and SQL and how to use these tools effectively..

Student B

I already had some experience with SQL and BI reporting, so the SQL part of the assignment felt quite natural. Most of the work there was translating the business questions into clear GROUP BY logic and checking that the joins followed the star schema correctly.

MDX, on the other hand, was completely new to me and much harder to get used to. Query building was often confusing, especially coming from SQL where you think in rows and columns instead of hierarchies and members. Understanding how filters work through subselects and how to use functions like TopCount or DESCENDANTS took some trial and error. The Atoti UI helped a lot, because I could build queries with drag-and-drop first and then inspect the generated MDX to learn the correct patterns.